# Introduction to Reinforcement Learning

## From Principles to Modern Methods

prof. dr. Laura Dioșan,
stud. Bogdan Miron

April 25, 2026

"Babes Bolyai" University of Cluj Napoca, Faculty of Mathematics and Computer Science

# Course outline

A Gentle Introduction

The Formal Framework: Markov Decision Processes

Value Functions and the Bellman Equation

Model-Free Reinforcement Learning

A Taxonomy of RL Agents

A Peek at Deep Reinforcement Learning

Wrap-up

# A Gentle Introduction

# The question that started it all

*Can a machine learn to solve a problem*

*not by being told what to do,*

*not by seeing labelled examples,*

**but by trying, failing, and trying again?**

This is the central question of **Reinforcement Learning**.

# March 2016 — Move 37

- AlphaGo plays a move no professional would consider.
- Commentators, live: "*That's not a human move.*"
- Fifty moves later, it turns out to be brilliant. AlphaGo wins.
- Nobody taught AlphaGo that move — it learned it by playing against itself.

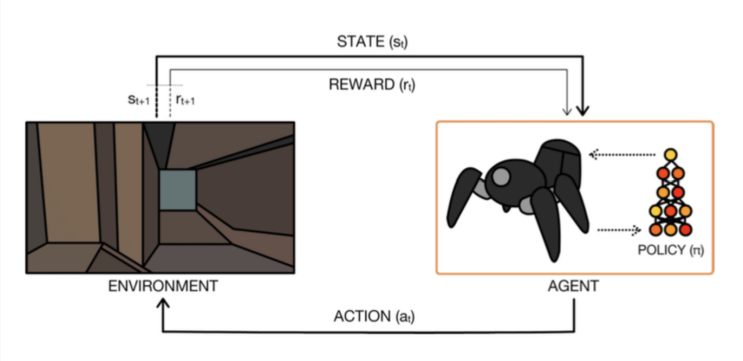
# What is reinforcement learning?

At its core, RL is a framework for learning by interaction.

**The paradigm in one sentence**
An **agent** takes **actions** in an **environment** and receives **rewards**; its goal is to learn a behaviour that maximizes total reward over time.

- No labels. No correct answers. Just a scalar reward signal.
- Actions shape the future: today's action changes tomorrow's data.
- Learning happens *through* acting, not before.

# The agent–environment loop




The agent observes a state, picks an action, receives a reward, the environment transitions to a new state — and the cycle repeats.

# Three things that make RL hard

1. Delayed rewards. A good move today may only pay off 50 moves later. Which action deserves the credit? (The **credit assignment problem**.)
2. Exploration vs. exploitation. Should I try something new that might be better, or stick with what I already know works?
3. No supervisor. The environment tells you *how good* an outcome was, not *what you should have done*. The feedback is *evaluative*, not *instructive*.

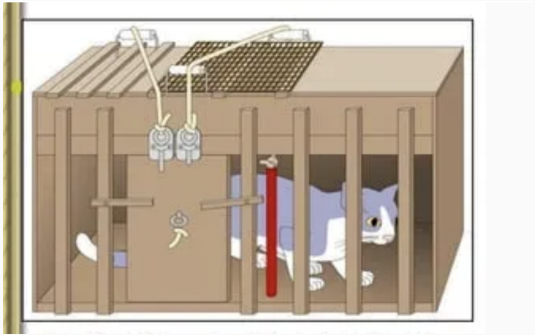
# RL sits at the confluence of three traditions

**Psychology** + **Control theory** + **Computer science**

- **Psychology** gives us the vocabulary: *reinforcement*, reward, behaviour.
- **Control theory** gives us the mathematics: dynamic programming, optimal control.
- **Computer science** gives us the algorithms: TD learning, Q-learning, deep RL.

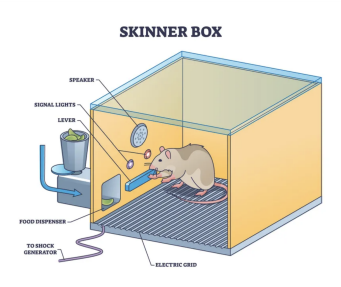
Let's trace each river.

# Psychology, part 1: Thorndike (1898)



- Edward Thorndike placed cats in puzzle boxes; they had to learn to press a lever to escape.
- He measured how the escape time decreased over repeated trials.
- **Law of Effect:** behaviours followed by satisfying consequences become *more likely*; those followed by discomfort, *less likely*.
- The first quantitative theory of learning from consequences.

# Psychology, part 2: Skinner's mouse




B. F. Skinner's **operant conditioning chamber** (1930s–1950s):

- Mouse presses a lever $\longrightarrow$ food pellet drops.
- Press again $\longrightarrow$ more food.
- The mouse learns the behaviour is *reinforced*.

**The origin of the name**

This is literally where the word *reinforcement* in "reinforcement learning" comes from.

# Control theory: Bellman (1957)

- Richard Bellman formalizes **dynamic programming**: breaking a sequential decision problem into overlapping subproblems.
- Introduces the Bellman equation — arguably the single most important equation in reinforcement learning.
- Also coins a phrase you already know: *"the curse of dimensionality."*

# Computer science: learning from experience

**1959** Arthur Samuel's checkers player learns by playing itself — one of the first machine learning systems of any kind.

**1988** Richard Sutton formalizes **Temporal-Difference learning**: learning from the difference between successive predictions.

**1992** Gerald Tesauro's **TD-Gammon** learns world-class backgammon from self-play — a 30-year preview of AlphaZero.

# The modern explosion

- **2013–15:** DQN masters Atari from raw pixels (DeepMind).
- **2016:** AlphaGo defeats Lee Sedol 4–1.
- **2017:** AlphaZero — superhuman chess, Go, shogi from zero knowledge.
- **2019:** MuZero learns *without being told the rules*.
- **2022:** RLHF makes ChatGPT useful to humans.
- **2024+:** RL for reasoning, robotics, scientific discovery.

# RL vs. the rest of machine learning

| | **Supervised** | **Unsupervised** | **Reinforcement** |
|---|---|---|---|
| Signal | Correct labels | No signal | Scalar reward |
| Data | i.i.d. dataset | i.i.d. dataset | Sequential, correlated |
| Feedback | Instructive | — | Evaluative |
| Who generates data? | Fixed | Fixed | The agent itself |
| Goal | Predict | Find structure | Act optimally |

**The key difference**

In RL, the agent's actions *change the data distribution it will see next.* This single fact makes RL fundamentally harder than supervised learning.

# Where RL is used today

Games · Robotics · Recommender systems · LLM alignment (RLHF)
Chip floorplanning · Data-center cooling · Finance · Drug discovery

# The Formal Framework: Markov Decision Processes

# From cartoon to mathematics

We now make the agent–environment picture precise.

The mathematical object is called a **Markov Decision Process (MDP)**, and it will be the playground for the entire course.

Almost every algorithm we will see is either solving an MDP, approximating a solution to one, or learning to solve an unknown one.

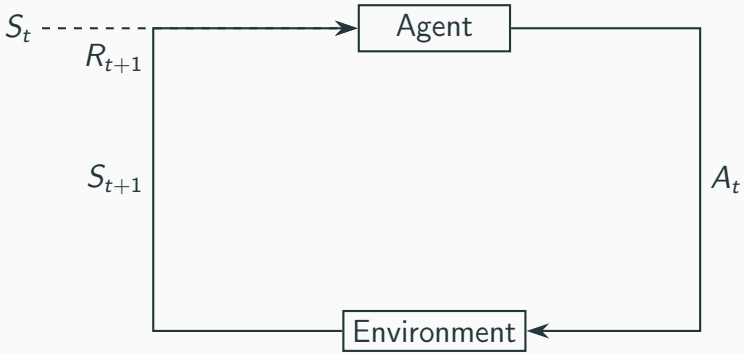
# Markov Decision Process: the 5-tuple

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$$

- $\mathcal{S}$ — the set of **states**
- $\mathcal{A}$ — the set of **actions**
- $P(s' \mid s, a)$ — the **transition dynamics**: probability of landing in $s'$ after taking $a$ in $s$
- $R(s, a)$ — the **reward function**: expected immediate reward
- $\gamma \in [0, 1]$ — the **discount factor**

Everything else in RL will be built on top of this.

# The interaction loop, formally




At each discrete time step $t = 0, 1, 2, \ldots$:

1. The agent observes a state $S_t \in \mathcal{S}$.
2. The agent selects an action $A_t \in \mathcal{A}$.
3. The environment emits a reward $R_{t+1} \in \mathbb{R}$ and a next state $S_{t+1}$.

# The Markov property

**Intuition**
*The future depends on the present — not on how we got here.*

**Formally**

$$\mathbb{P}(S_{t+1}, R_{t+1} \mid S_t, A_t) \;=\; \mathbb{P}(S_{t+1}, R_{t+1} \mid S_t, A_t, S_{t-1}, A_{t-1}, \ldots, S_0)$$

- The current state captures all information relevant for the future.
- This is an *assumption* — often approximated, rarely exactly true.
- When violated, we enrich the state or add memory (RNNs, transformers).

# Policies

A **policy** $\pi$ is the agent's way of choosing actions.

**Deterministic**

$$\pi : \mathcal{S} \to \mathcal{A}$$

$$a = \pi(s)$$

"In state $s$, do $a$."

**Stochastic**

$$\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s)$$

"In state $s$, take $a$ with probability $\pi(a \mid s)$."

- Stochastic policies are strictly more general.
- Essential for exploration and for partially observed problems.

# Return and the discount factor

The **return** is the total accumulated reward from time $t$ onward:

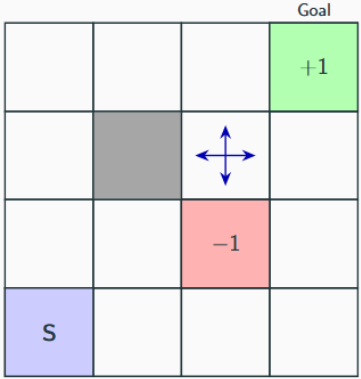
$$G_t \;=\; R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \;=\; \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

**Why $\gamma$?**

- **Mathematical.** Guarantees the sum converges in infinite horizons.
- **Economic.** *A reward now is worth more than the same reward later.*
- **Epistemic.** Far-future outcomes are more uncertain — discount them.

Typical values: $\gamma \in \{0.9, 0.95, 0.99\}$. $\gamma = 0$: myopic. $\gamma \to 1$: far-sighted.

# Running example: Gridworld




Actions:
$\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$

- States: 16 grid cells. Actions: $\{\uparrow, \downarrow, \leftarrow, \rightarrow\}$.
- Reward: $+1$ at the goal, $-1$ at the trap, 0 otherwise. $\gamma = 0.9$.

# Value Functions and the Bellman Equation

# The core question

*How good is it to be in this state?*

*How good is it to take this action here?*

To answer, we introduce **value functions** —
the quantities RL algorithms spend most of their time estimating.

# The state-value function

For a policy $\pi$:

$$\boxed{V^{\pi}(s) \;=\; \mathbb{E}_{\pi}\big[\, G_t \,\big|\, S_t = s \,\big]}$$

**In words**
"If I start in state $s$ and act according to $\pi$ forever after,
what total discounted reward do I expect to collect?
How good is it to be in this state, assuming I'll keep following my current strategy?"

Every policy induces its own value function.
We often compare policies by comparing their value functions.

# Comparing policies

**Value functions are policy-dependent.**
Each policy $\pi$ induces its own function $V^\pi : \mathcal{S} \to \mathbb{R}$. The same state can be highly valuable under a smart policy and nearly worthless under a random one — the state hasn't changed, only the strategy has.

**Policy ordering.** We say $\pi_1$ is at least as good as $\pi_2$ if

$$\boxed{\pi_1 \;\geq\; \pi_2 \qquad \Longleftrightarrow \qquad V^{\pi_1}(s) \;\geq\; V^{\pi_2}(s) \;\text{ for all } s \in \mathcal{S}}$$

This is a *partial* order: some pairs of policies are incomparable (better in some states, worse in others).

**The goal of RL.** Find an *optimal policy* $\pi^*$ whose value function dominates every other:

$$\boxed{V^*(s) \;=\; \max_\pi V^\pi(s) \quad \text{for all } s \in \mathcal{S}}$$

**Existence (finite MDPs)**
There always exists at least one *deterministic* policy $\pi^*$ that is optimal in every state *simultaneously* — so the partial order does have a maximum.

# The action-value function

$$\boxed{Q^\pi(s,a) \;=\; \mathbb{E}_\pi\big[\, G_t \,\big|\, S_t = s,\, A_t = a \,\big]}$$

**In words**
"If I'm in state $s$, take action $a$ *right now*,
and then follow $\pi$ thereafter — what do I expect to get?"

**Why $Q$ matters more than $V$ in practice**
If we know $Q^\pi(s,a)$ for every action, we can *act* without knowing the environment's dynamics — just pick the action with the highest $Q$.

# The action-value function — in words

**Plain English**
"If I'm in state $s$, take action $a$ *right now*, and then follow $\pi$ thereafter — what do I expect to get?"

The key difference from $V^\pi$:

- $V^\pi(s)$ — "how good is this *state* if I follow $\pi$ from now on?"
- $Q^\pi(s, a)$ — "how good is it to take *this specific action* in this state, then follow $\pi$?"

So $Q$ lets us evaluate actions *individually* — even ones the current policy wouldn't normally take.

The two are tightly linked:

$$\boxed{V^\pi(s) \;=\; \sum_{a \in \mathcal{A}} \pi(a \mid s)\, Q^\pi(s, a)}$$

The state-value is just the policy-weighted average of the action-values.

# Why $Q$ matters more than $V$ in practice

Suppose we're in state $s$ and want to pick the best action.

**Using $V^\pi$:** we'd have to compute

$$a^* \;=\; \arg\max_a \sum_{s'} P(s' \mid s, a)\,\big[\,R(s, a, s') + \gamma V^\pi(s')\,\big]$$

This requires knowing:

- the transition probabilities $P(s' \mid s, a)$ — where actions take us,
- the reward function $R(s, a, s')$ — what we get for each transition.

**Using $Q^\pi$:** the expectation is already baked in. To act, just compute

$$\boxed{a^* \;=\; \arg\max_{a \in \mathcal{A}} Q^\pi(s, a)}$$

**The payoff: model-free RL**
An agent can learn $Q$ from raw experience — trying actions, observing rewards — and then act optimally without ever building a model of the world. This is the foundation of **Q-learning**, **SARSA**, and **DQN**.

# The Bellman equation: the idea

The return satisfies a simple recursion:

$$G_t \;=\; R_{t+1} + \gamma\, G_{t+1}$$

Take expectations under $\pi$, condition on $S_t = s$:

$$V^\pi(s) \;=\; \mathbb{E}_\pi\big[R_{t+1} + \gamma\, V^\pi(S_{t+1}) \,\big|\, S_t = s\big]$$

**The key insight**
The value of a state equals the expected immediate reward *plus* the discounted value of where you land next.

# The Bellman expectation equation

Expanding the expectation over actions and next states:

$$V^\pi(s) \;=\; \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \left[ R(s,a) + \gamma\, V^\pi(s') \right]$$

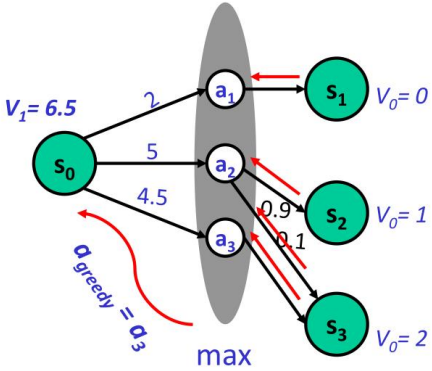
And for the action-value function:

$$Q^\pi(s,a) \;=\; \sum_{s'} P(s' \mid s, a) \left[ R(s,a) + \gamma \sum_{a'} \pi(a' \mid s')\, Q^\pi(s', a') \right]$$

These are *linear* systems in $V^\pi$ / $Q^\pi$. In principle, solvable by linear algebra.

# Bellman equation

## Bellman Backup




$Q_1(s,a_1) = 2 + \gamma\, 0$
$\sim 2$

$Q_1(s,a_2) = 5 + \gamma\, 0.9 \sim 1$
$+ \gamma\, 0.1 \sim 2$
$\sim 6.1$

$Q_1(s,a_3) = 4.5 + \gamma\, 2$
$\sim 6.5$

# Optimal value functions

Define the **optimal** value functions by taking the best over all policies:

$$V^*(s) \;=\; \max_{\pi} V^{\pi}(s), \qquad Q^*(s,a) \;=\; \max_{\pi} Q^{\pi}(s,a)$$

An **optimal policy** $\pi^*$ achieves these values. It is obtained greedily from $Q^*$:

$$\pi^*(s) \;=\; \arg\max_{a} \; Q^*(s,a)$$

**A remarkable fact**
For any finite MDP, there always exists a *deterministic* optimal policy. Randomness is never required to be optimal — when the MDP is fully observed.

# The Bellman optimality equation

$$V^*(s) = \max_a \sum_{s'} P(s' \mid s, a) \left[ R(s, a) + \gamma V^*(s') \right]$$

$$Q^*(s, a) = \sum_{s'} P(s' \mid s, a) \left[ R(s, a) + \gamma \max_{a'} Q^*(s', a') \right]$$

- The max is what makes this nonlinear.
- No closed-form solution in general.
- But we can solve it *iteratively* — and this is exactly what RL algorithms do.

# Back to Gridworld

| | | | |
|---|---|---|---|
| 0.73 → | 0.81 → | 0.90 → | +1 |
| 0.66 ↑ | | 0.81 ↑ | 0.90 ↑ |
| 0.59 ↑ | 0.53 ↓ | −1 | 0.81 ↑ |
| 0.53 → Start | 0.59 → | 0.66 → | 0.73 ↑ |

- Each cell's number is $V^*(s)$.
- Values decrease as the optimal path to the goal gets longer — each extra step multiplies the return by $\gamma$.
- The arrows show $\pi^*$: the action with the highest $Q^*(s, a)$.

# If we knew the MDP...

When $P$ and $R$ are known, the Bellman optimality equation can be solved directly by **dynamic programming**:

**Value iteration**
Repeat until convergence:

$$V(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)\left[R + \gamma V(s')\right]$$

**Policy iteration**
Alternate until convergence:

1. Evaluate $V^\pi$
2. Improve: $\pi' = \arg\max_a Q^\pi$

**But...**
We almost never know $P$ and $R$ in the real world.
So we must *learn* them — or sidestep them entirely.

# Model-Free Reinforcement Learning

# Two ways to not know the model

**Model-based RL**
Learn $\hat{P}$ and $\hat{R}$ from experience, then *plan* using the learned model.

*Pro:* sample-efficient.
*Con:* model errors compound.

**Model-free RL**
Skip the model. Learn $V$, $Q$, or $\pi$ *directly* from experience.

*Pro:* simple, robust.
*Con:* needs lots of data.

Most of what we'll see next is **model-free**.

# Monte Carlo methods

**Idea:** run a full episode, then update values using the actual return $G_t$.

$$V(S_t) \leftarrow V(S_t) + \alpha \left[ G_t - V(S_t) \right]$$

- *Unbiased:* we use the true return observed in the episode.
- *High variance:* $G_t$ depends on every decision until the episode ends.
- Only works for *episodic* tasks — we need episodes to end.

Monte Carlo is what you do when you wait until the game is over to learn from it.

# Temporal-Difference learning: the big idea

**Bootstrapping**

**Update a guess using another guess.**

Instead of waiting for the full return $G_t$, estimate it as

$$R_{t+1} + \gamma\, V(S_{t+1}) \qquad \text{and use that.}$$

# TD(0) — the simplest TD update

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{\left[ R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \right]}_{\text{TD error } \delta_t}$$

- Update *online*, after every single step — no need to wait for the episode to end.
- The TD error $\delta_t$ measures *surprise*:
  "how much better (or worse) was this step than I predicted?"
- If predictions are perfect, $\mathbb{E}[\delta_t] = 0$ — updates stop.

# The unified picture: DP, MC, TD

| | **Bootstrap?** | **Sample?** |
|---|---|---|
| Dynamic Programming | Yes | No (needs model) |
| Monte Carlo | No | Yes |
| Temporal-Difference | Yes | Yes |

# SARSA — on-policy TD control

Apply TD learning to $Q$:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[ R_{t+1} + \gamma \, Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$

- The name comes from the quintuple used:
  **S**tate, **A**ction, **R**eward, next **S**tate, next **A**ction.
- Uses the action $A_{t+1}$ *actually taken* by the current policy.
- Learns $Q^\pi$ for the policy being followed $\implies$ **on-policy**.

# Q-learning — off-policy TD control

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[ R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right]$$

**The one-character difference**

SARSA uses $Q(S_{t+1}, A_{t+1})$ — the action *actually taken*.

Q-learning uses $\max_{a'} Q(S_{t+1}, a')$ — the *best* action available.

- Learns $Q^*$ regardless of the policy generating the data $\implies$ **off-policy**.
- Watkins (1989). Convergence guarantees under mild conditions. Foundation of DQN.

# On-policy vs. off-policy, intuitively

**On-policy (SARSA):**

"Learn the value of the policy I'm *actually* running — including its exploration mistakes."

*Safer.* Accounts for the fact that exploration has cost.

**Off-policy (Q-learning):**

"Learn the value of the *optimal* policy, even while exploring suboptimally."

*Greedier.* Can reuse data from any source.

Classic illustration: the cliff-walking task.
SARSA walks safely along the top; Q-learning takes the short, risky route.

# The explore / exploit dilemma

*Do I go to my favourite restaurant,*
*or try the new one across the street?*

- **Exploit:** use what you know to maximize reward *now*.
- **Explore:** try something uncertain to learn, possibly doing better later.
- Pure exploitation gets stuck in local optima; pure exploration never profits.
- Every RL algorithm needs a strategy for this trade-off.

# $\varepsilon$-greedy: the workhorse

$$A_t = \begin{cases} \arg\max_a Q(S_t, a) & \text{with probability } 1-\varepsilon \\ \text{a uniformly random action} & \text{with probability } \varepsilon \end{cases}$$

- Most of the time, act greedily with respect to current $Q$.
- Occasionally, try something at random.
- Often decay $\varepsilon$ over time: explore a lot early, exploit later.

**Other strategies worth knowing**
Optimistic initialization · softmax / Boltzmann exploration · UCB · Thompson sampling · intrinsic motivation / curiosity

# Q-learning on Gridworld

- Start with $Q \equiv 0$.
- Act $\varepsilon$-greedily; after every step, update $Q$ using the Q-learning rule.
- After enough episodes, $Q \to Q^*$ and the greedy policy becomes optimal.

# A Taxonomy of RL Agents

# What does the agent actually learn?

An RL agent can learn any combination of:

- A value function ($V$ or $Q$) — evaluates states or state-action pairs.
- A policy $\pi$ — chooses actions directly.
- A model ($\hat{P}$, $\hat{R}$) — predicts the environment.

The combination you pick defines a *family* of methods.

# The agent zoo

- **Value-based:** learn $Q$, act greedily. *E.g. Q-learning, DQN.*
- **Policy-based:** learn $\pi$ directly. *E.g. REINFORCE.*
- **Actor–critic:** learn both $\pi$ (actor) and $V$ or $Q$ (critic). *E.g. A2C, PPO, SAC.*
- **Model-based:** learn the environment, then plan. *E.g. Dyna, MuZero.*

# A Peek at Deep Reinforcement Learning

# When tabular methods break

Everything so far assumed we can store one number per $(s, a)$ pair.

- Atari from pixels: $|\mathcal{S}| \approx 256^{210 \times 160 \times 3}$. Unthinkable.
- Continuous control: state is real-valued — uncountable.
- Go: $|\mathcal{S}| \approx 10^{170}$.

**The answer**
Replace the table $Q(s, a)$ with a function approximator $Q_\theta(s, a)$
— typically a **neural network**.

# Function approximation, in one slide

Represent the value function with learnable parameters $\theta$:

$$Q_\theta(s, a) \;\approx\; Q^*(s, a)$$

Train by minimizing a TD-like loss:

$$\mathcal{L}(\theta) \;=\; \mathbb{E}\Big[\,\big(\,R + \gamma \max_{a'} Q_\theta(S', a') - Q_\theta(S, A)\,\big)^2\,\Big]$$

- The network *generalizes* across similar states.
- But combining bootstrapping + off-policy learning + function approximation — the "*deadly triad*" — can cause divergence.
- Making this stable is most of what Deep RL research has been about.

# DQN: Deep Q-Networks (Mnih et al., 2013, 2015)

- Input: raw game pixels. Output: one $Q$-value per action.
- Q-learning + CNN + two stabilizing tricks:
  - **Experience replay.** Store transitions, sample random minibatches to break correlations.
  - **Target network.** Use a slowly-updated copy of $\theta$ when computing the bootstrap target.
- First agent to reach human-level on dozens of Atari games *from pixels alone*.

# Policy gradient methods

**Idea:** parameterize $\pi_\theta(a \mid s)$ directly and optimize it by gradient ascent.

The REINFORCE gradient:

$$\nabla_\theta J(\theta) \;=\; \mathbb{E}_{\pi_\theta}\big[\,\nabla_\theta \log \pi_\theta(A_t \mid S_t) \;\cdot\; G_t\,\big]$$

**In one sentence**
Increase the (log-)probability of actions that led to high returns; decrease it for actions that led to low ones.

- Handles continuous action spaces naturally.
- High variance — hence baselines, critics, trust regions.

# Actor–critic: the best of both worlds

Two networks, learning together:

**Actor** $\pi_\theta$

Picks actions.
Updated by policy gradient.

**Critic** $V_\phi$ or $Q_\phi$

Evaluates actions.
Updated by TD learning.

The critic reduces the variance of the actor's gradient;
the actor tells the critic which states to focus on.

**Modern workhorses**
A2C / A3C · PPO (the one most people use) · SAC · DDPG / TD3

# RLHF: why you've heard of RL lately

1. Start with a pretrained language model.
2. Collect human preferences over pairs of model responses.
3. Train a **reward model** to predict which response a human would prefer.
4. Fine-tune the LLM with **PPO**, using the reward model as the reward signal.

**The insight**

When you can't write down a reward function, *learn* one from human judgment — then apply standard RL.

# Wrap-up

# Key takeaways

1. RL is about learning **from interaction**, guided by a scalar reward.
2. The formal object is the **MDP**; everything builds on it.
3. The **Bellman equation** is the recursive backbone of every value-based method.
4. **TD learning** lets us learn online, without a model, by bootstrapping.
5. **Q-learning**, **policy gradients**, and **actor–critic** are the three main algorithmic families — and they power everything from Atari to ChatGPT.

# Open challenges

- **Sample efficiency.** Deep RL often needs millions of interactions.
- **Exploration** in sparse-reward environments.
- **Generalization** to unseen tasks and environments.
- **Safety** and robustness — especially with learned rewards.
- **Sim-to-real** transfer for robotics.
- **Offline RL:** learning from fixed datasets, without interaction.

# Resources to go deeper

- **Book:** Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed., free online). The reference.
- **Lectures:** David Silver's UCL course (YouTube) — gold standard.
- **Practical:** OpenAI's *Spinning Up in Deep RL*.
- **Code:** *Gymnasium* (environments); *Stable-Baselines3* and *CleanRL* (readable PyTorch implementations).
- **Landmark papers:** DQN (Mnih et al. 2015), PPO (Schulman et al. 2017), AlphaZero (Silver et al. 2017), InstructGPT (Ouyang et al. 2022).